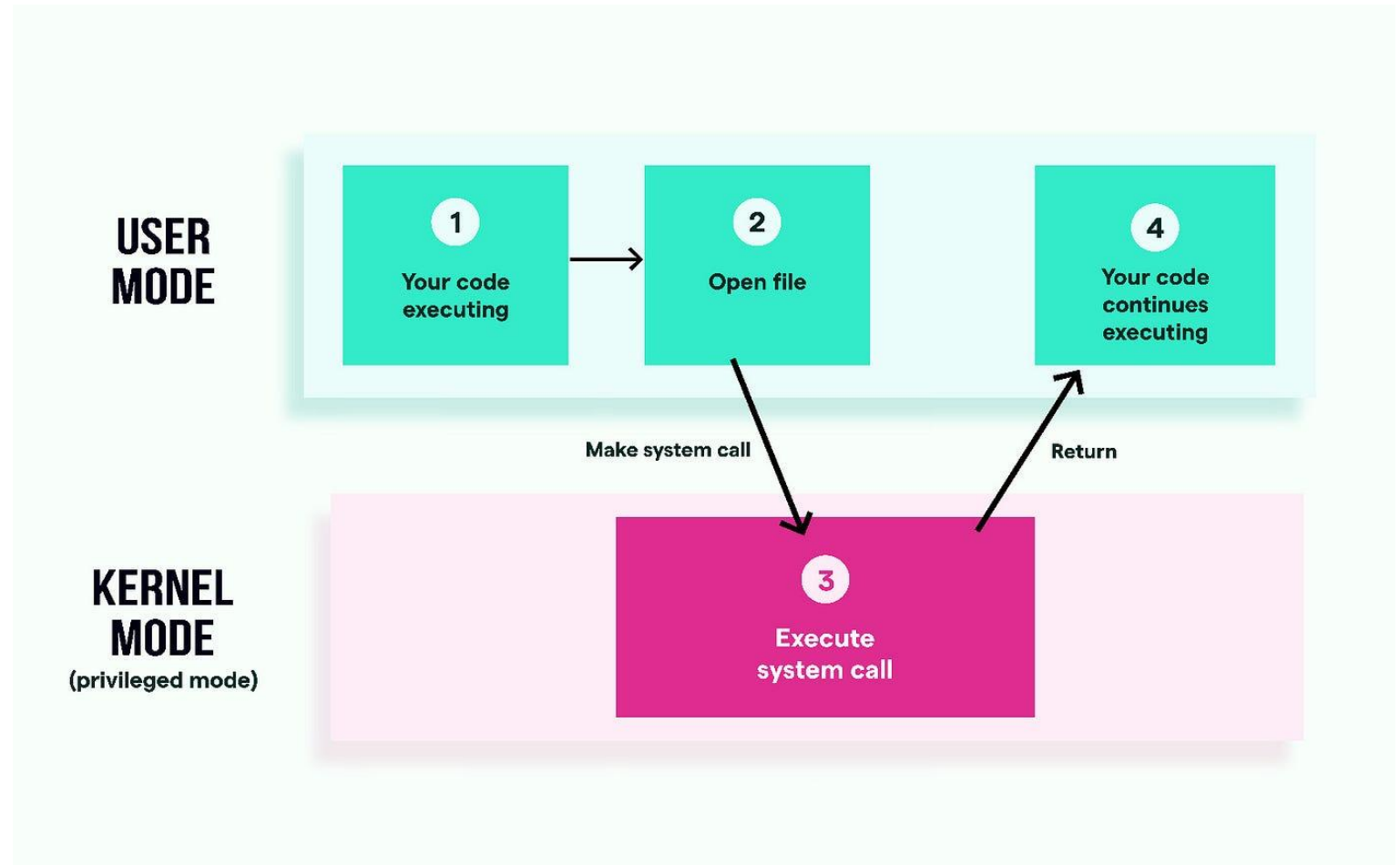


Remember
this?



Why can't user programs run with full control over the system?

Operating System Operation

By : Bilal Ahmed

Learning Objectives

1

Understand the dual mode operation in OS.

2

Differentiate between kernel and user mode.

3

Analyze the mode switching between user and kernel mode.

4

Witness the different system calls for different OSs.

Need for Dual Mode Operation



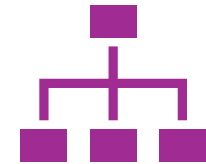
Protect the system from
faulty or malicious programs



Prevent unauthorized
access to hardware



Ensure system stability and
security



Separate user-level activities
from system-level
operations

User Mode

Mode in which **user applications execute**

Has **limited access** to system resources

Cannot execute privileged instructions

Examples:

- Text editors
- Web browsers
- Media players

Characteristics of User Mode

01

Restricted
instruction set

02

Cannot
directly access
hardware

03

Must request
OS services
using **system
calls**

04

Ensures safety
and isolation
between
processes

Kernel Mode

Mode in which the **operating system kernel** executes

Has **full access** to hardware and system resources

Can execute privileged instructions

Responsible for:

- Process management
- Memory management
- Device control

Characteristics of Kernel Mode



Executes critical
system operations



Direct hardware access
allowed



High privilege level



Errors can affect the
entire system

User mode vs kernel mode

Feature	User Mode	Kernel Mode
Access level	Limited	Full
Hardware access	Not allowed	Allowed
Privileged instructions	Not permitted	Permitted
Risk level	Low	High

Mode Switching

01

Occurs when a **system call** is invoked

02

CPU switches from user mode to kernel mode

03

OS performs requested service

04

Control returns to user mode after completion

Importance of Mode Switching

Maintains system security

Prevents misuse of system resources

Ensures controlled execution of programs

Summary



User mode protects the system from application errors



Kernel mode enables full system control



Dual mode operation is essential for security and stability



System calls act as the controlled gateway between user mode and kernel mode.

Flow of a System Call

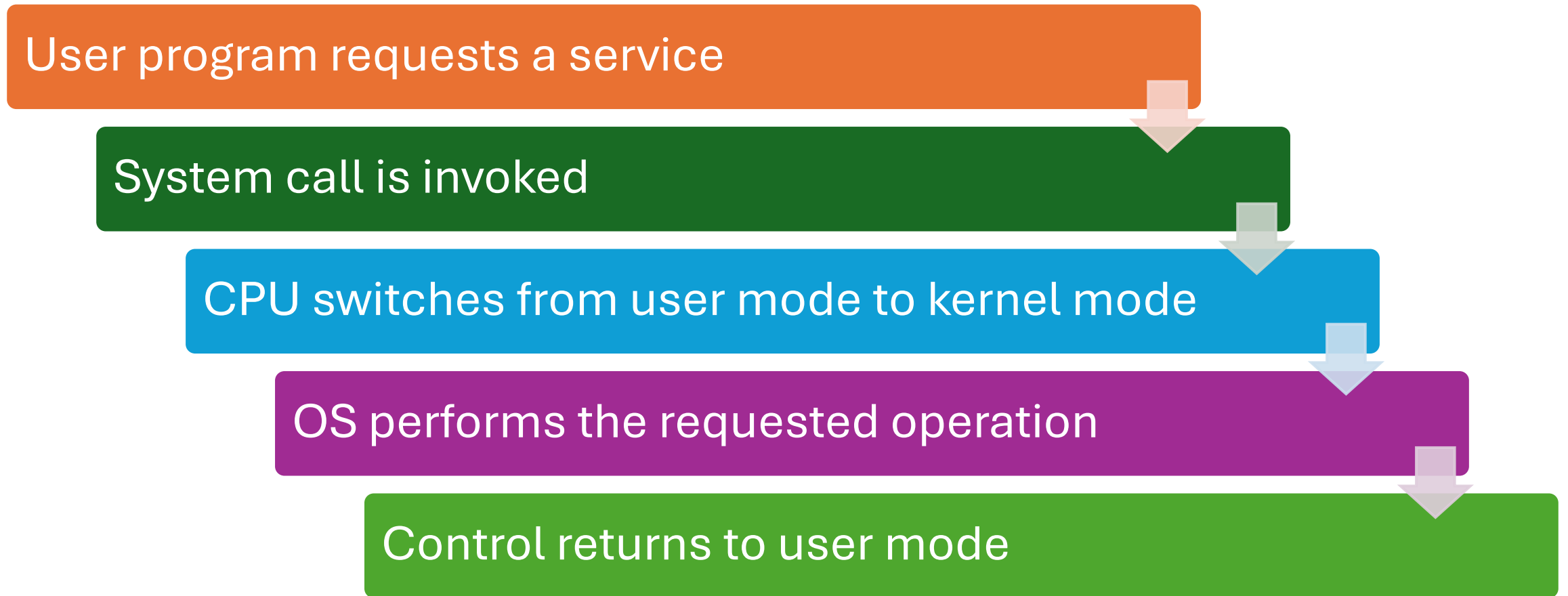
User program requests a service

System call is invoked

CPU switches from user mode to kernel mode

OS performs the requested operation

Control returns to user mode





Do all system calls
perform the same
function, or are they
grouped by purpose?



Into what groups can system calls be classified based on their functionality?

Types of system calls



PROCESS
CONTROL



FILE
MANAGEMENT



DEVICE
MANAGEMENT



INFORMATION
MAINTENANCE



COMMUNICATION

Process Control System Calls

Examples:

- Create a process
- Terminate a process
- Load and execute a program
- Wait for process completion

Purpose:

- Manage process lifecycle
- Control CPU execution

Examples

UNIX / Linux	Windows
fork()	CreateProcess()
wait()	WaitForSingleObject()
exit()	ExitProcess()

File Management System Calls

Examples:

- Create and delete files
- Open and close files
- Read and write files
- Change file permissions

Purpose:

- Provide secure file access
- Manage persistent storage

Examples

UNIX / Linux	Windows
open()	CreateFile()
read()	ReadFile()
write()	WriteFile()
close()	CloseHandle()

Device Management System Calls

Examples:

- Request or release devices
- Read from or write to devices
- Get or set device attributes

Purpose:

- Coordinate hardware usage
- Ensure correct I/O operations

Examples

UNIX / Linux	Windows
read()	ReadConsole()
write()	WriteConsole()
ioctl()	SetConsoleMode()

Information Maintenance System Calls

Examples:

- Get system time and date
- Get process or system status
- Set system parameters

Purpose:

- Monitor system state
- Maintain system configuration

Information Maintenance System Calls

UNIX / Linux	Windows
getpid()	GetCurrentProcessId()
time()	GetSystemTime()
stat()	GetFileInformationByHandle()

Communication System Calls

Examples:

- Create communication channels
- Send and receive messages
- Share memory between processes

Purpose:

- Enable cooperation between processes
- Support concurrent execution

Communication System Calls

UNIX / Linux	Windows
<code>pipe()</code>	<code>CreatePipe()</code>
<code>Shm_open()</code>	<code>CreateFileMapping()</code>
<code>send()</code> / <code>recv()</code>	<code>SendMessage()</code> / <code>RecvMessage()</code>

What is relationship between OS services and system calls?

Relationship Between OS Services and System Calls



OS services define *what* the operating system offers



System calls define *how* these services are accessed



Every OS service is implemented using **one or more system calls**

OS Services vs System Calls

OS Services	System Calls
High-level functionality	Low-level interface
What OS provides	How OS services are requested
Conceptual	Implementation mechanism

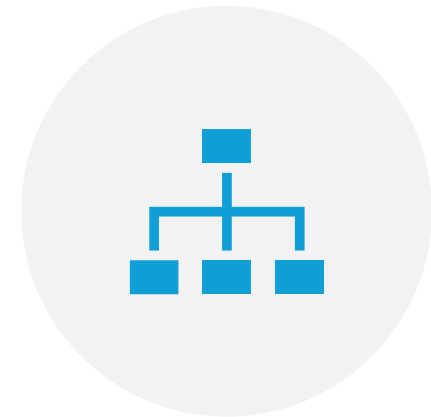
Summary



SYSTEM CALLS ACT AS A **BRIDGE**
BETWEEN APPLICATIONS AND THE OS



THEY PROVIDE **SECURE AND
CONTROLLED ACCESS** TO OS
SERVICES



THEY ARE ESSENTIAL FOR **PROCESS
EXECUTION, RESOURCE
MANAGEMENT, AND PROTECTION**